EmberDB: A FHIR-Native Time-Series Storage Engine for Continuous Patient Monitoring

Abhinav Srivastava
CTO, Ambra
Abhinav@ambra911.com

***Abstract*—ICU monitors sample vital signs every few seconds. The standard interoperability format for those readings is HL7 FHIR Observation. General-purpose time-series databases (InfluxDB, TimescaleDB) ingest the data but discard the clinical semantics at the storage layer, forcing reconstruction at query time. EmberDB is a Rust time-series engine for FHIR Observation data. Records carry a composite metric key of patient identifier, LOINC code, and unit. Data is partitioned into 1-hour chunks and persisted through a write-ahead log. Five clinical pattern-detection algorithms run inside the engine: CUSUM changepoint detection, PELT segmentation, seasonal decomposition, Mahalanobis anomaly scoring, and moving-window analysis. On a synthetic MIMIC-IV workload (500 patients, 48-hour ICU stays, 1.7M chartevents) EmberDB sustains 3.2M records/sec in memory mode with sub-microsecond point queries. Against a SQLite baseline on patient-scoped workloads it is 7-9× faster on ingestion and 8,890× faster on point lookup.**

***Index Terms*—FHIR, time-series database, patient monitoring, ICU, clinical informatics**

I. Introduction

ICU bedside monitors sample vital signs (heart rate, blood pressure, oxygen saturation, respiratory rate, temperature) at intervals from one second to a few minutes. Across a hospital that adds up to millions of observations per patient-day [1]. HL7 FHIR has become the standard wrapper for these readings. Each measurement is an `Observation` resource carrying a LOINC-coded type, a numeric value with units, a patient reference, and a timestamp [3].

Storing those observations efficiently *without losing the clinical structure* is the open problem. InfluxDB [4] and TimescaleDB [5] can absorb the throughput. What they cannot do is preserve the structured relationship between patient, observation code, and clinical context. Both flatten FHIR into generic tag/value pairs. PostgreSQL preserves schema but has no time-partitioned storage and needs expensive joins to answer temporal queries.

EmberDB is a Rust time-series storage engine built around continuous patient monitoring. Four design decisions distinguish it.

FHIR-native storage. Observations carry composite metric keys encoding patient ID, LOINC code, and unit. Nothing has to map back to FHIR after ingestion.

Time-partitioned architecture. Data is bucketed into configurable time chunks (one hour by default). Range scans and chunk-level retention fall out naturally.

Built-in clinical pattern detection. CUSUM changepoint, PELT segmentation, seasonal decomposition, multivariate Mahalanobis anomaly detection, and moving-window analysis all run inside the engine.

Write-ahead log persistence. The WAL has chunk-level durability markers, so crash recovery does not require sacrificing ingestion performance.

We evaluate EmberDB on synthetic MIMIC-IV chartevents and compare it to a SQLite baseline on identical workloads.

II. Background and Related Work

A. Clinical time-series data

Clinical streams behave differently from IoT or infrastructure metrics in three ways that matter for storage. They are multivariate and physiologically correlated (heart rate, blood pressure, and SpO2 do not move independently). They carry semantic baggage that must be preserved end-to-end: each value has a LOINC code, a patient reference, and clinical context. And the sampling is irregular, with measurement frequency a function of patient acuity and clinical protocol rather than a fixed cadence.

MIMIC-IV [2] is the canonical reference dataset for these properties. The `chartevents` table runs to hundreds of millions of rows across thousands of ICU stays, with item IDs mapping to specific physiological measurements.

B. Existing storage approaches

InfluxDB organizes data by measurement name with tag-based indexing. The design fits infrastructure monitoring well. For FHIR Observations it requires flattening into measurement/tag/field triples, which throws away the structured patient/code/value relationship.

TimescaleDB adds time-partitioned hypertables to PostgreSQL. The relational structure survives, but row-based storage overhead and SQL parsing remain on the hot path for high-frequency point lookups.

SQLite is a common embedded choice for clinical applications. Its simplicity is the selling point. The lack of time-aware partitioning and the need for manual index management on temporal queries are the cost.

OpenTSDB and **QuestDB** are purpose-built time-series stores. Both, like InfluxDB, treat every metric as a generic nu-

meric stream with no first-class notion of healthcare resource models.

III. System Design

A. Architecture overview

EmberDB is written in Rust. It exposes both a library API and a REST interface (built on `warp`). Internally there are three layers: an in-memory chunk manager, a write-ahead log for durability, and a JSON chunk persistence layer.

B. FHIR-native metric encoding

Each FHIR Observation becomes an internal `Record` with a composite metric name:

```
metric_name = "{patient_id}|{loinc_code}|{unit}"
```

A heart rate for patient P1234 ends up stored as `P1234|8867-4|/min`. The encoding lets the engine answer both patient-scoped and code-scoped queries by prefix matching on the key, with no secondary indexes.

Multi-component observations (blood pressure being the canonical case, with systolic and diastolic) are decomposed into records sharing a prefix. The FHIR component structure is preserved, and each component can be analysed as its own time series.

C. Time-partitioned chunks

Records live inside `TimeChunk` structures, each spanning a configurable duration (one hour by default). A chunk contains a `HashMap<String, Vec<Record>>` mapping metric names to sorted record vectors, a resource-type index mapping FHIR resource types to their constituent metric names, and a small block of metadata (creation time, record count, dirty flag). The chunk boundary for any timestamp is computed by flooring to the nearest multiple of the chunk duration, which keeps chunk identification at $O(1)$.

D. Write-ahead log

The WAL is what gives EmberDB crash recovery. Each record gets persisted before it lands in the in-memory chunk. Records are serialised as JSON with a 4-byte length prefix. A chunk that crosses 10,000 records or 1 MB gets flushed, and the matching WAL entries are marked durable at that point. On recovery the engine loads the persisted chunks from disk and replays whatever WAL entries are left.

E. MIMIC-IV data loader

A MIMIC-IV loader module ships with EmberDB. It maps chartevent item IDs to FHIR LOINC codes (itemid 220045 $\rightarrow$ LOINC 8867-4 for heart rate, and so on) and turns rows into `Record` structures directly. The path is zero-copy, so there is no overhead from building intermediate FHIR JSON.

IV. Pattern Detection

EmberDB provides five built-in pattern detection algorithms, configured via a TOML file and operating directly on stored records.

A. CUSUM changepoint detection

The Cumulative Sum (CUSUM) algorithm maintains running sums S^+ and S^- tracking positive and negative deviations from the series mean:

$$S_i^+ = \max(0,\ S_{i-1}^+ + (x_i - \mu - k))$$

$$S_i^- = \max(0,\ S_{i-1}^- + (\mu - k - x_i))$$

where $k = 0.5\sigma$ is the sensitivity parameter. A changepoint is declared when either sum exceeds the threshold $h = \theta\sigma$, where θ is configurable (default 2.0).

B. PELT segmentation

The Pruned Exact Linear Time (PELT) algorithm finds the optimal segmentation by minimizing the penalized cost:

$$\sum_{j=1}^{m+1} \mathcal{C}(y_{\tau_{j-1}+1:\tau_j}) + m \cdot \beta$$

where $\mathcal{C}$ is the Gaussian negative log-likelihood cost for each segment and β is the penalty term. Changepoints with magnitude below $\theta\sigma$ are pruned.

C. Seasonal decomposition

Time series are decomposed into trend T_t, seasonal S_t, and residual R_t components using moving-average smoothing. Both additive ($Y_t = T_t + S_t + R_t$) and multiplicative ($Y_t = T_t \times S_t \times R_t$) models are supported.

D. Mahalanobis anomaly detection

For multivariate anomaly detection across correlated vitals, EmberDB computes the Mahalanobis distance:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Points exceeding a configurable threshold (default 3.0) are flagged as outliers. Metric groups can be specified explicitly or auto-detected via Pearson correlation.

E. Moving window analysis

A sliding window computes per-window statistics (trend slope, volatility, or range) and flags windows whose statistic deviates from the population by more than θ standard deviations.

V. Evaluation

We evaluate EmberDB on three benchmark suites: standalone performance, comparative analysis against SQLite, and a synthetic MIMIC-IV workload. All experiments were run on an Apple Silicon machine running macOS, with EmberDB compiled in release mode (`--release`).

A. Standalone performance

1) Write throughput: Table I shows write throughput across batch sizes with persistence disabled (memory-optimized mode).

2) Query latency: Table II reports average query latency over 100K ingested records (100 patients, single metric).

TABLE I
EMBERDB WRITE THROUGHPUT (MEMORY MODE)

Records	Throughput (rec/s)
1,000	2,689,372
10,000	1,467,926
50,000	2,582,106
100,000	3,238,495
500,000	3,297,006

TABLE II
EMBERDB QUERY LATENCY (μS)

Query Type	Avg Latency (μs)
Point (narrow range)	0.1
Latest value	0.9
Patient + code	126.2
Full patient (prefix)	13.6
Time range (50K span)	61.1

3) Pattern detection: Table III reports per-invocation latency for each detection algorithm on 2,000 records.

4) WAL performance: With persistence enabled (fsync per record), EmberDB sustains 124 records/s write throughput due to disk synchronization overhead. Recovery from WAL (10,000 records) completes in 0.008 seconds.

5) Storage overhead: At 100,000 records with full persistence, EmberDB uses 149.6 bytes per record on disk (14.96 MB total) due to JSON serialization overhead.

B. Comparative analysis: EmberDB vs. SQLite

1) Write throughput: Table IV compares write throughput. EmberDB's in-memory mode avoids disk synchronization overhead, while SQLite uses WAL mode with `PRAGMA synchronous=NORMAL`.

2) Query latency: Table V compares query latency across four query types.

TABLE III
PATTERN DETECTION LATENCY (μS)

Algorithm	Avg Latency (μs)
CUSUM changepoint	598.0
Seasonal decomposition	5,167.5
Moving window	423.5
Mahalanobis (multivariate)	251.5

TABLE IV
WRITE THROUGHPUT COMPARISON (REC/S)

Records	EmberDB	SQLite	Ratio
1,000	738,257	246,038	3.0×
10,000	1,258,139	247,915	5.1×
50,000	2,145,535	282,572	7.6×
100,000	2,197,044	263,857	8.3×
500,000	2,170,299	254,397	8.5×

TABLE V
QUERY LATENCY COMPARISON (μS)

Query	EmberDB	SQLite	Speedup
Point lookup	0.1	764.5	8,890×
Latest value	0.9	857.4	953×
Time range	63.1	808.9	12.8×
Full patient	128.2	912.6	7.1×

C. MIMIC-IV synthetic workload

We generated synthetic MIMIC-IV chartevents for 500 patients, each with a 48-hour ICU stay and 6 vital signs sampled every 5 minutes (576 measurements per vital per patient). Vital sign distributions were calibrated to published ICU ranges: HR $\mathcal{N}(84, 17)$, SBP $\mathcal{N}(121, 23)$, DBP $\mathcal{N}(70, 14)$, SpO2 $\mathcal{N}(96.8, 2.8)$, RR $\mathcal{N}(18, 4)$, Temp $\mathcal{N}(98.6, 1.2)$ °F. Each chartevent was mapped to a FHIR Observation via LOINC code and ingested into both EmberDB and SQLite.

1) Ingestion: The dataset contains 1,728,000 chartevents (500 patients × 6 vitals × 576 measurements). EmberDB ingested the FHIR-converted records at 1,466,955 records/sec (1.18s total). SQLite achieved 160,980 records/sec (10.73s), a 9.1× speedup for EmberDB.

2) Query performance: Table VI reports query latency on the MIMIC workload.

TABLE VI
MIMIC-IV QUERY LATENCY (μS)

Query	EmberDB	SQLite	Speedup
Single vital (1h)	3.2	30.8	9.5×
Full patient stay	944.6	616.7	0.65×
Cohort vital (1h)	852.4	16,773.6	19.7×
Latest vital	27.1	79.7	2.9×

VI. DISCUSSION

FHIR-native advantages. By encoding patient ID and LOINC code directly into the metric key, EmberDB removes the need for secondary indexes or joins when retrieving patient-scoped observations. This design works well for clinical dashboard queries that retrieve all vitals for a single patient over a shift or stay.

Pattern detection at the storage layer. Embedding changepoint detection and anomaly scoring directly in the storage engine avoids data export overhead and allows real-time alerting pipelines that operate on the same data path as persistence.

Limitations. EmberDB currently serializes chunks as JSON, which has higher storage overhead than columnar formats (149.6 bytes/record vs. ∼107 bytes/record for SQLite at 100K records). On the full-patient-stay query in the MIMIC workload, SQLite outperforms EmberDB (617 μs vs. 945 μs) because EmberDB requires separate queries per vital metric while SQLite returns all vitals in a single index scan. The single-writer architecture limits concurrent write throughput.

The pattern detection algorithms use simplified implementations; production deployment would need validated statistical libraries.

Future work. Planned improvements include columnar chunk encoding (e.g., Parquet), concurrent chunk writers with sharded locking, integration with FHIR Subscription for push-based alerts, and support for FHIR Bulk Data export.

VII. Conclusion

EmberDB is evidence that you do not have to choose between high ingest throughput and native clinical semantics. Encode patient identity and LOINC codes into the storage key, partition the data into time-aligned chunks, integrate the pattern-detection algorithms directly, and the result handles continuous patient monitoring without giving up time-series performance. On synthetic MIMIC-IV workloads it is competitive with SQLite, with richer query semantics and no post-hoc data transformation.

References

[1] A. E. W. Johnson, T. J. Pollard, L. Shen, et al., "MIMIC-III, a freely accessible critical care database," *Scientific Data*, vol. 3, p. 160035, 2016.

[2] A. E. W. Johnson, L. Bulgarelli, L. Shen, et al., "MIMIC-IV, a freely accessible electronic health record dataset," *Scientific Data*, vol. 10, p. 1, 2023.

[3] HL7 International, "FHIR Release 4: Observation Resource," https://hl7.org/fhir/R4/observation.html, 2019.

[4] InfluxData, "InfluxDB: Open Source Time Series Database," https://www.influxdata.com/, 2024.

[5] Timescale Inc., "TimescaleDB: An Open-Source Time-Series SQL Database," https://www.timescale.com/, 2024.